Choosing a Model

Unit 13 (optional): When a More Complicated Model Earns Its Keep

Stat 220 | Statistics for Data Science

Where we left off

- Unit 2 read a slope. Unit 3 built models with many predictors and judged them honestly. Unit 4 put error bars on a single prediction.
- Every one of those units assumed somebody had already handed you the model.
- Nobody ever said how that choice gets made.

Principle: the gap this unit fills

You know how to fit a model and how to check whether it is any good. This unit is about the step before both of those: deciding which model to reach for, and being able to say why.

The question nobody answered for you

- "Should I use linear regression or a random forest?" is the most common real question in applied work, and the least often taught.
- The honest answer is never "the fancy one" and never "the simple one." It depends on four things you can actually check: your question, your outcome, your sample size, and what it costs you to be wrong.
- This unit turns that into something you can do out loud, in order, in an interview or a meeting.

In practice: the shape of this unit

"We have 300 rows, 40 candidate predictors, and we need to know whether the price change hurt retention. What would you fit?"

What this unit gives you

Things to know

- the four questions a model can be asked, and that they are separate
- bias and variance as the reason the error curve is U-shaped
- what actually moves the best complexity: n, noise, and the number of candidates
- which model families answer which question well
- what each family costs you in interpretability and in data

Mistakes to stop making

- reaching for a flexible model on 40 rows
- reaching for a simple model on 400,000 and calling it caution
- using a prediction model's coefficients as effects
- choosing the model after seeing which one gave the answer you wanted
- reporting the winner of a long search as if it were the only thing you tried

The four questions a model gets asked

1. **Is X important?** One variable, one verdict.
2. **Which variables are important?** A list, out of many candidates.
3. **What is the predicted value?** A number for a new case.
4. **How certain am I?** An honest range around any of the above.

Principle: they are not the same question

These four look like one job because a single `fit()` call seems to answer all of them at once. It does not. A model can be excellent at question 3 and actively misleading on question 1.

One model, four different grades

Same 392 cars, same random forest predicting mpg.

Question	Grade	Why
Is weight important?	poor	Importance scores split credit among correlated predictors. Weight and displacement steal from each other.
Which variables matter?	fair	It ranks them, but the ranking shifts every refit and says nothing about direction.
What is the predicted mpg?	strong	This is what the model optimizes.
How certain am I?	fair	You can get an interval from held-out errors, but not from the model itself.

Principle: pick the tool for the question you were actually asked

The forest is not a bad model. It is a bad answer to question 1.

Your turn #1

Your turn: four teams, four questions

A hospital has 5 years of records on 90,000 patients and 200 variables. Four people walk in with four requests.

1. Legal: "Does our new intake form change how long patients wait?"
2. Ops: "Which 10 of these 200 things should we even be tracking?"
3. The app team: "Given a patient at check-in, how long will they wait?"
4. The director: "How sure are you about any of this?"

For each one, say what you would fit. Two of these want the same model family for very different reasons. One of them is not a modeling question at all until you ask something first.

Your turn #1: answer

1. **Legal wants one coefficient defended.** Regression with the intake form as the predictor of interest and the known confounders included on purpose. You need a slope with a standard error, not the best prediction. And ask first whether the form was rolled out in a way that lets you claim anything causal (Unit 4).
2. **Ops wants a short list from 200 candidates.** Regularization, the lasso, which shrinks most coefficients to zero and keeps a handful. Report it as a screening list, not as "these 10 cause wait times."
3. **The app wants a number.** Anything that predicts well. With 90,000 rows you can afford a forest or boosting, and nobody needs to read its coefficients.
4. **The director wants uncertainty.** Not a model at all. Ask which of the first three she means, because the honest interval is different for each one.

Principle:

Questions 1 and 2 both use regression, and for opposite reasons: one defends a single coefficient, the other throws most of them away.

Simple or complicated, the honest version

- The question is not "which model is better." Asked that way it has no answer.
- The question is: **do I have enough data to afford this much flexibility?**
- Flexibility is something you buy. The price is paid in sample size, and if you cannot afford it you do not get a slightly worse model, you get a much worse one.

Principle: the reframe

A complicated model is not a stronger claim about the world. It is a bigger bet that you have enough data to pin down what you just allowed to vary.

Two ways to be wrong

Too simple

- Wrong in the same direction every time.
- Fit it on fresh data and you get almost the same wrong answer.
- Stable, and stably off.

(**bias**)

Too complicated

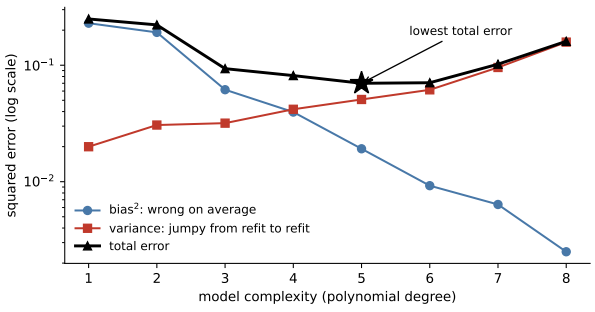
- Right on average across many refits.
- But any single fit chases the noise in the rows you happened to draw.
- Refit on new data and it looks like a different model.

(**variance**)

Principle: why the error curve is U-shaped

Adding flexibility trades one for the other. Bias falls, variance rises, and the total is worst at both ends. The whole art is finding where the trade stops paying.

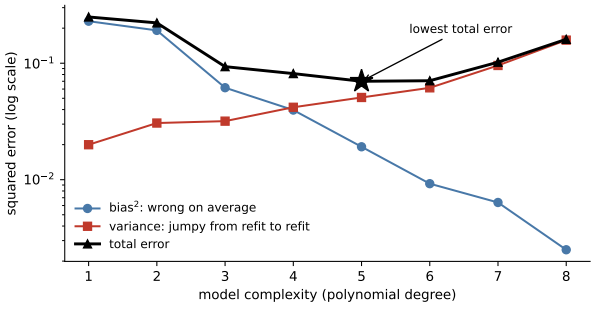
The U-shape, taken apart



600 datasets of $n = 60$, each fit with polynomials of every degree, compared against the true curve we simulated.

- Bias falls the whole way. More flexibility really does get closer to the truth on average.
- Variance climbs the whole way.
- Their sum bottoms out at degree 5.

The U-shape, taken apart

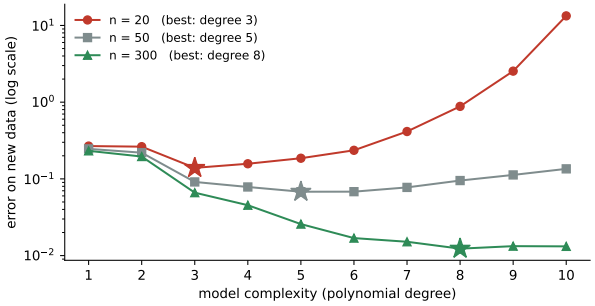


600 datasets of $n = 60$, each fit with polynomials of every degree, compared against the true curve we simulated.

- Bias falls the whole way. More flexibility really does get closer to the truth on average.
- Variance climbs the whole way.
- Their sum bottoms out at degree 5.

Neither curve has a minimum. Only the sum does.

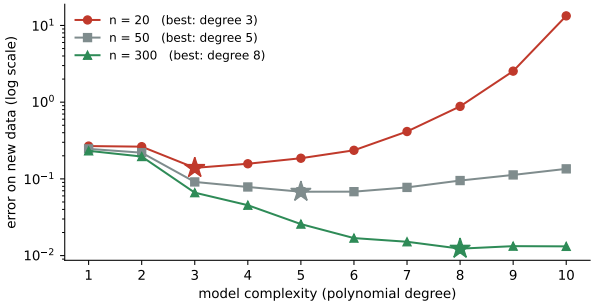
Can you afford the truth?



The true curve never changes. The only thing that changes is how many rows you were given.

- $n = 20$: best is degree **3**.
- $n = 50$: best is degree **5**.
- $n = 300$: best is degree **8**.

Can you afford the truth?



The true curve never changes. The only thing that changes is how many rows you were given.

- $n = 20$: best is degree **3**.
- $n = 50$: best is degree **5**.
- $n = 300$: best is degree **8**.

Overshooting to degree 10 costs you a factor of **96** at $n = 20$, and **7 percent** at $n = 300$. Same mistake, same model, opposite verdict.

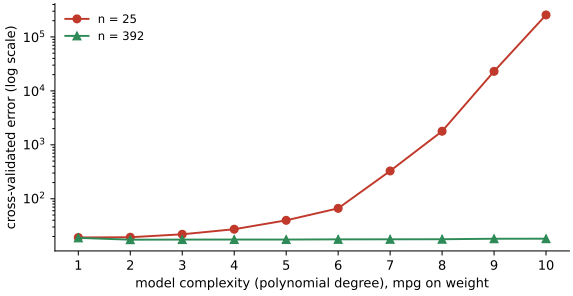
What moves the best complexity

When this goes up	Best model gets	Because
Sample size n	more complex	variance is the part data buys down
Noise in the outcome	simpler	more of what you would fit is noise
Candidate predictors p	simpler	searching is itself a form of fitting
Smoothness of the truth	simpler	there is less structure to find
Cost of a confident error	simpler	stable models fail less spectacularly
Need to explain a coefficient	simpler	you cannot defend what you cannot read

Principle: the one-line version

More rows push you toward flexibility. Noise, candidates, and the need to explain push you back.

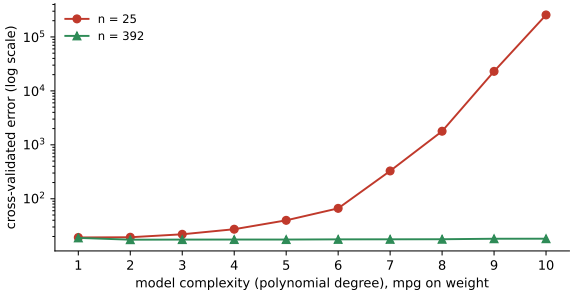
The same question on real cars



mpg on weight, cross-validated, real data.

- The best degree hardly moves. The relationship really is close to a simple curve.
- What collapses is the **penalty for guessing too high**.
- At $n = 25$, overshooting to degree 10 is a catastrophe. At $n = 392$ it is nearly free.

The same question on real cars



mpg on weight, cross-validated, real data.

- The best degree hardly moves. The relationship really is close to a simple curve.
- What collapses is the **penalty for guessing too high**.
- At $n = 25$, overshooting to degree 10 is a catastrophe. At $n = 392$ it is nearly free.

With enough data you are allowed to be wrong about complexity. With 25 rows you are not.

Your turn #2

Your turn: same model, two companies

Two teams both want to predict whether a customer churns next month. Both propose the identical gradient-boosted model with 500 trees.

- Team A works at a bank with 2.4 million customers and 60 clean variables.
- Team B works at a startup with 340 customers and 60 variables, most of them hand-entered.

Same model, same code, same question. Is one of them making a mistake? Say which, and say what they should do instead. Then say what would have to change for your answer to flip.

Your turn #2: answer

- **Team A is fine.** With 2.4 million rows the variance cost of 500 trees is tiny, and that is exactly the regime where flexibility is cheap. They should still hold out a test set and still refuse to read the importances as effects.
- **Team B is not.** 340 rows and 60 hand-entered variables is the worst corner of the table: small n, large p, high noise. A boosted forest there will fit the data beautifully and predict new customers badly.
- **What B should do:** a regularized logistic regression on a handful of variables chosen for a reason, cross-validated, with the one-standard-error rule to break ties toward the simpler fit.
- **What would flip it:** B getting to a few tens of thousands of clean rows. Not a better algorithm. More data.

Principle:

The model was never the problem. The ratio of what you are estimating to what you were given is.

Training rank and test rank disagree

- The training leaderboard is almost perfectly ordered by degree. Highest degree wins.
- The test leaderboard is close to reversed at the top, and the winner is usually a team that picked 2, 3, or 4.
- Nobody who picked 9 is anywhere near the top, and their curve is the one that looked best ten minutes ago.

Principle: the point of committing first

You had to choose without seeing the test data. That is the real situation, always. Cross-validation is how you make that choice honestly, using only the data you were allowed to look at.

You may be asked:

"How did you pick the complexity?" has exactly one good answer, and it is not "it fit best."

How to read a model catalog

Ask three things, in this order. The first two narrow it to a family. The third picks within it.

1. **What kind of outcome is it?** A number, a yes/no, a count, a category, a time until something happens. This is not negotiable and it eliminates most of the catalog.
2. **Which of the four questions am I answering?** Explaining one effect, screening many, predicting, or quantifying uncertainty.
3. **How much data do I have relative to what I am estimating?** This sets how much flexibility you can afford.

Trap: the usual order

Most people start at step 3 with "what is the strongest algorithm," which is the only one of the three that cannot be answered on its own.

When the outcome is a number

Reach for	When	What it costs you
Linear regression	you must defend a coefficient	assumes the shape is roughly a line
Regression with transforms or interactions	the shape bends, and you can say how	you have to specify the bend yourself
Lasso or ridge	many candidates, prediction focus	coefficients are shrunk, so no clean effects
A single tree	you want a rule someone can follow	unstable, and coarse at the edges
Forest or boosting	prediction is all that matters	no readable coefficients, needs a lot of rows

Principle:

Going down this table buys flexibility and spends interpretability. There is no row that gives you both.

When the outcome is not a number

- **Yes or no.** Logistic regression is the default, and everything above still applies: regularize when there are many candidates, use a forest when only prediction matters. The extra job is that the output is a probability and somebody has to pick a threshold. **Unit 10.**
- **A count, or a rate per unit of exposure.** Do not force it into a linear model on the raw count. Reason from how the count is generated first. **Units 6 and 7.**
- **A category with more than two levels.** Same logic, more bookkeeping.

Principle: the outcome decides first

The type of outcome is the one part of this choice that is not a judgment call. Get it wrong and nothing downstream can rescue you.

The two-model habit

- Fit the simplest defensible model **and** the flexible one. Always both.
- If they agree, you have a finding and you report the simple one, because it is easier to defend and easier to maintain.
- If they disagree, you have learned something specific: there is structure the simple model is missing, and now you go find out what it is.

Principle: the disagreement is the result

A gap between a linear fit and a forest is not a reason to ship the forest. It is a lead. Usually it is an interaction or a nonlinearity you can name, put into the simple model, and then explain.

In practice: a strong thing to say

"The forest beat the regression by four points, so I looked for what it was using, found the threshold at 30 days, added it, and the regression caught up. I shipped the regression."

Your turn #3

Your turn: choose, and say why in one sentence each

1. 1,100 loan applications, 12 clean variables, and the regulator will ask you to justify every rejection.
2. 6 million ad impressions, 400 variables, and the only thing that matters is click rate.
3. 60 clinical trial patients, one treatment variable, and the question is whether the drug works.
4. 25,000 rows of sensor data where you know the relationship is a curve but not which curve.

For each, name the family and the reason. The reason is the part that gets graded.

Your turn #3: answer

1. **Logistic regression, probably regularized.** Not because it predicts best, but because "justify every rejection" is question 1 and a coefficient is the only output you can defend to a regulator.
2. **Boosting or a forest.** 6 million rows makes flexibility cheap, nobody will read a coefficient, and click rate is pure question 3.
3. **A t-test or a regression with the treatment indicator.** With $n = 60$ and one variable of interest, anything flexible would spend your data estimating things nobody asked about. Randomization is doing the hard work here, not the model.
4. **A flexible fit, then go find the curve.** A forest or a spline to see the shape, then name it and put it in a regression you can explain. This is the two-model habit.

Principle:

Three of these four are decided before you look at the data at all.

Choosing the model that gave you the answer you wanted

Trap: the most common way this goes wrong

You fit a regression and the effect is not significant. You try a forest. You add an interaction. You drop two outliers. Somewhere in there the effect appears, and that is the model that goes in the report.

- Every one of those steps is defensible on its own. The sequence is not.
- The p-value on the final model is computed as if it were the only model you fit. It was the twelfth.
- The fix is not to stop exploring. It is to **decide the model before you look at the answer**, and to report what you did if you changed your mind.

In practice: a strong thing to say

"I picked the specification before running it. Here is the one I preregistered and here are the three others I tried afterward, with the same conclusion."

Two more that look like competence

Trap: reporting the winner of a search

You cross-validated 40 model configurations and report the best one's cross-validated error as if it were an unbiased estimate. It is not. It is the maximum of 40 noisy numbers, and it is optimistic. Keep a test set you touch once, at the end.

Trap: complexity as signaling

"We used a neural network" sounds like effort. On 800 rows it is a claim that you had enough data to fit one, and you did not. The most senior person in the room is the one most likely to ask why you did not start with a regression.

Principle:

Simplicity you chose on purpose reads as judgment. Simplicity you defaulted to reads as inexperience. The difference is whether you can say what you ruled out.

LLM check: mistake or not? (1 of 2)

LLM check: you asked for help choosing a model

"With 200 patients and 45 predictors, I'd recommend a random forest. It handles nonlinearity and interactions automatically, is robust to outliers, and gives you feature importances so you can see which variables drive the outcome. You can then report the top predictors as the key risk factors."

Three separate problems. Find them before the next slide.

LLM check: answer

- **The ratio.** 200 rows and 45 predictors is the small-n, large-p corner. A forest there will fit well and generalize poorly. Regularized regression is the honest choice.
- **"Automatically" is doing a lot of work.** Handling interactions automatically means spending data to discover them. With 200 rows you cannot afford the search.
- **The last sentence is the real error.** Feature importance is not an effect. Reporting the top predictors as risk factors turns a prediction tool into a causal claim, which is the exact mistake Unit 3 warned about and Unit 4 explains.

Principle:

The answer is not wrong about what a forest does. It is wrong about whether this dataset can pay for one, and about what the output means.

Choosing a model

1. **Which of the four questions is this?** Explain one effect, screen many, predict, or quantify uncertainty.
2. **What type is the outcome?** Number, yes/no, count, category. This eliminates most of the catalog and is not a judgment call.
3. **Count what you have against what you are estimating.** Rows, candidate predictors, and how noisy the outcome is.
4. **Fit the simplest defensible model first.** It is your baseline and often your answer.
5. **Fit a flexible one too, on the same folds.** Cross-validate both the same way.
6. **If they are within one standard error, take the simpler one.** If they are not, find out what the flexible one is using.

The traps

Trap: the ones that bite most often

- Reaching for flexibility you do not have the rows to pay for.
- Choosing the model after seeing which one gave the answer you wanted.
- Reporting the winner of a search as an unbiased estimate.
- Reading feature importances, or shrunk coefficients, as effects.
- Answering question 3 when you were asked question 1.
- Treating a count or a yes/no as if it were a number.
- Defaulting to simple without being able to say what you ruled out.

Ten questions to be able to answer

1. Linear regression or a random forest? What do you need to know before answering?
2. Why does more data let you afford a more complicated model?
3. Name the two ways a model can be wrong, and which one more data fixes.
4. Why does the error curve have a minimum when neither of its two parts does?
5. You have 300 rows and 80 predictors. What do you rule out immediately, and why?
6. Your forest beats your regression by five points. What do you do next?
7. Why is the cross-validated error of your best-of-40 model optimistic?
8. When is a simpler model the better choice even though it predicts worse?
9. What does the one-standard-error rule say, and why is it not just laziness?
10. Somebody hands you feature importances and calls them risk factors. Respond.

Mastery checklist

You have this unit when you can, from memory:

- name the four questions and say which model families answer each one well
- explain bias and variance without using the words bias or variance
- say what pushes the best complexity up and what pushes it down
- look at n, p, and the noise level and rule out families on the spot
- run the two-model habit and act on the disagreement
- defend a simple model as a choice rather than a default

What you should be able to do with software

Nobody is asking you to memorize syntax. You are expected to know *which* procedure the question calls for, what to hand it, and what the output means. With an AI assistant open, you should be able to produce any of these and defend the result.

You should be able to	**What you would reach for**
Score a range of complexities	loop a hyperparameter, `cross_val_score` each
Compare two families honestly	the same `KFold` object for both
Apply the one-standard-error rule	CV mean, then `scores.std()/sqrt(k)`
Screen many candidates	`LassoCV`, then read what survived
Ask whether more data would help	`learning_curve`, and see if it is still falling
Keep an honest final estimate	one held-out test set, scored once
Find what the flexible model found	partial dependence, then name the pattern

If you cannot say what the output means, the fact that you produced it is worth nothing.

Where this fits

- This unit is optional and it is deliberately a crossroads. It uses Unit 2's coefficients, Unit 3's cross-validation, and Unit 4's honest error bars, and it points forward to logistic regression in **Unit 10** and count models in **Units 6 and 7**.
- Nothing here is a new technique. It is the reasoning that decides which technique you were going to use anyway.

Principle: the through-line

Anyone can fit the flexible model. The judgment worth having is knowing what your data can pay for, and being able to say so out loud before you see the answer.